PROYECTO 3 · SPRINT 3

INTELLIGENT DOCUMENT ASSISTANT

DOCUMENTO DE ESTUDIO

Cómo se busca el archivo

La analogía del mapa semántico: cada fragmento recibe una coordenada, y una pregunta encuentra las coordenadas más cercanas.

FICHA DE BÚSQUEDA #003

Modelo	**multilingual-e5-small**
Dimensiones	**384**
Prefijos	**passage / query**
Métrica	**distancia coseno**
Búsqueda	**exacta**
Respuesta LLM	**Todavía no**
Total	**Sprint 3**

El archivo ya sabe cortar. Ahora aprende a ubicar.

Next.js 16 · TypeScript · PostgreSQL + pgvector · Transformers.js

Cada fragmento conserva su documento y su página.

LA IDEA CENTRAL

El archivo ya sabe cortar; ahora aprende a ubicar

En el Sprint 1 el archivo aprendió a leer: recibió un PDF, extrajo el texto y guardó cada página con su número. En el Sprint 2 aprendió a recortar: dividió cada página en fragmentos trazables. En el Sprint 3 damos el paso que faltaba: cada fragmento recibe una **coordenada numérica**, llamada **embedding**, y una pregunta puede encontrar las coordenadas más cercanas.

La pregunta del sprint ya no es «¿cómo cortamos?». Es anterior a la respuesta generativa: **¿cómo encontramos los fragmentos que hablan de lo mismo que la pregunta, aunque no repitan las mismas palabras?** Todavía no le pedimos al modelo que redacte nada. Primero construimos el bibliotecario; el redactor llega en el Sprint 4.

> **La tesis del Sprint 3:** un embedding no es una palabra clave ni una traducción. Es una coordenada en un espacio construido por el modelo. Dos textos que hablan de temas parecidos quedan cerca, y la distancia coseno nos dice cuánto.

El reparto de roles

EN EL MAPA	EN LA APLICACIÓN	DÓNDE VIVE
Ficha temática	El fragmento que ya existe desde el Sprint 2	`DocumentChunk`
Coordenada	El vector de 384 números del fragmento	`ChunkEmbedding.embedding`
Modelo	E5-small, con sus prefijos y su pooling	`lib/embeddings/provider.ts`
Regla de comparación	La distancia coseno entre dos vectores	Operador `<=>` de pgvector
Filtro de compatibilidad	Modelo, revisión y dimensión usados	`ChunkEmbedding`
Búsqueda	Los fragmentos más cercanos a una pregunta	`lib/embeddings/search.ts`
Mostrador	La pantalla de diagnóstico	`/search`
Respuesta generativa	Todavía no existe	Sprint 4 en adelante

EL MAPA

Qué es un embedding, sin magia

Un embedding es una lista de números que representa un texto. No es una descripción que podamos leer ni una traducción: es una posición en un espacio de muchas dimensiones. El modelo aprende a ubicar textos con significados parecidos en lugares parecidos.

LO QUE ES

Una coordenada semántica

«El contrato se puede cancelar con treinta días de aviso» y «Para rescindir el acuerdo hay un plazo de un mes» pueden quedar cerca aunque no compartan casi palabras.

LO QUE NO ES

Una palabra clave

Una búsqueda por palabra clave exige coincidencia exacta. Un embedding compara significado: puede encontrar una cláusula relevante aunque la pregunta use sinónimos o cambie la redacción.

384 números, no un string

Para este proyecto, cada fragmento se convierte en un vector de **384 dimensiones**. No podemos leer esos números como una frase, pero sí compararlos. La dirección del vector es lo que importa; dos vectores que apuntan hacia el mismo lado representan ideas cercanas.

```
"El período de garantía del producto es de dos años."
→ [0.13, -0.08, 0.41, …]  ← 384 números

"La garantía cubre el producto durante 24 meses."
→ [0.11, -0.05, 0.39, …]  ← otra coordenada cercana

"El pago debe realizarse dentro de los primeros diez días."
→ [0.02, 0.27, -0.18, …]  ← una zona más lejana
```

Los prefijos de E5

E5 fue entrenado con dos formatos de entrada: `passage:` para los documentos y `query:` para las preguntas. No son adornos. Le indican al modelo qué tipo de texto está viendo y ayudan a que la comparación funcione como fue entrenada.

```
embedPassages(["El contrato se puede cancelar…"])
  → "passage: El contrato se puede cancelar…"

embedQuery("¿Cómo cancelo el contrato?")
  → "query: ¿Cómo cancelo el contrato?"
```

Si cambiás el prefijo, cambiás el vector. El modelo no ve un texto neutro: ve el texto con el formato con el que fue entrenado. Omitir `passage:` o `query:` no rompe la aplicación, pero degrada la calidad de la recuperación.

LA MÉTRICA

Similitud coseno y distancia coseno

La forma más común de comparar dos embeddings es medir el ángulo entre ellos. La similitud coseno vale 1 cuando los vectores apuntan en la misma dirección, 0 cuando son perpendiculares y −1 cuando apuntan en direcciones opuestas.

```
similitud(A, B) = (A · B) / (||A|| · ||B||)

Si A y B están normalizados, ||A|| = ||B|| = 1:
similitud(A, B) = A · B
```

Ejemplo con dos dimensiones:

```
A = [1, 0]
B = [0.9, 0.1]

A · B = 1 × 0.9 + 0 × 0.1 = 0.9
||A|| = 1
||B|| = √(0.9² + 0.1²) ≈ 0.9055
similitud ≈ 0.9 / 0.9055 ≈ 0.994
```

Los vectores apuntan casi en la misma dirección. En cambio, `[1, 0]` y `[0, 1]` son perpendiculares y su similitud es 0.

La distancia es el complemento

pgvector no devuelve similitud: devuelve **distancia coseno** con el operador `<=>`. La relación es simple:

```
distancia = 1 − similitud

similitud 1.0  → distancia 0.0   ← misma dirección
similitud 0.0  → distancia 1.0   ← perpendicular
similitud −1.0 → distancia 2.0   ← dirección opuesta
```

Por eso la búsqueda ordena por distancia **ascendente**: la distancia más chica es el fragmento más parecido. En la interfaz mostramos la distancia y también `1 − distancia` como porcentaje, para que sea más fácil de leer.

CONCEPTO	VALOR	QUÉ SIGNIFICA
Similitud	1	Los vectores apuntan en la misma dirección: textos muy parecidos.
Similitud	0	Vectores perpendiculares: sin relación semántica evidente.
Distancia	0	El fragmento es idéntico o casi idéntico a la consulta.
Distancia	1	No hay una dirección compartida; el resultado es poco relevante.
Distancia	2	Direcciones opuestas. En la práctica, muy lejos.

El operador `<=>` en pgvector

```
SELECT
  c.id,
  c.content,
  p."pageNumber",
  e.embedding <=> $1::vector AS distance
FROM "ChunkEmbedding" e
JOIN "DocumentChunk" c ON c.id = e."chunkId"
JOIN "DocumentPage" p ON p.id = c."documentPageId"
WHERE e."modelId" = $2
  AND e."modelRevision" = $3
  AND e.dimensions = $4
ORDER BY e.embedding <=> $1::vector ASC
LIMIT $5;
```

> **Menor distancia, mayor similitud.** La búsqueda no aplica un umbral en este sprint: devuelve los `topK` más cercanos aunque estén lejos. El umbral se decidirá cuando midamos la calidad con preguntas de evaluación.

LA DECISIÓN LOCAL

El modelo corre en tu máquina

Para empezar, el proyecto usa `Transformers.js` con el modelo multilingüe `Xenova/multilingual-e5-small`. La inferencia ocurre en el proceso de Node.js: el texto de tus documentos no viaja a un proveedor externo para generar embeddings.

NAVEGADOR

1. «Generar embeddings»

La isla cliente hace `POST /api/documents/[id]/embeddings`. Solo indica si quiere completar o regenerar.

↓

FRONTERA CLIENTE / SERVIDOR

SERVIDOR · ROUTE HANDLER

2. Validar id y cuerpo

El id debe tener un formato seguro. El cuerpo acepta `{ "regenerate": true | false }`. Un cuerpo inválido devuelve
400
.

↓

SERVIDOR · PROVEEDOR

3. Cargar el modelo una vez

`EmbeddingProvider` carga el pipeline de forma lazy y lo reutiliza. En desarrollo, el singleton vive en `globalThis` para sobrevivir a recargas de módulos.

↓

SERVIDOR · INFERENCIA

4. Generar el vector

Cada texto recibe su prefijo. El modelo aplica mean pooling y normalización L2. El resultado debe tener exactamente 384 números finitos.

↓

POSTGRESQL · PGVECTOR

5. Guardar la coordenada

El vector se envía como parámetro y se castea con `::vector`. Se guarda con el modelo, la revisión, la dimensión y el chunk.

Carga lazy y una sola instancia

La primera llamada descarga el modelo y sus archivos de tokenización en `.cache/transformers/`. Las llamadas siguientes reutilizan la instancia cargada. Si la carga falla, la promesa se limpia para permitir un reintento; no se cachea el error para siempre.

```
// Simplificado: la idea del singleton
let providerPromise = null;

export function getEmbeddingProvider() {
  if (!providerPromise) {
    providerPromise = loadExtractor()
      .then((extractor) => createEmbeddingProvider(extractor, model))
      .catch((error) => {
        providerPromise = null;
        throw error;
      });
  }

  return providerPromise;
}
```

Qué se guarda con cada vector

DATO	POR QUÉ IMPORTA
`chunkId`	Une el vector con el fragmento y su página. Es la trazabilidad del Sprint 2.
`modelId`	Identifica el modelo que generó el vector. No se comparan vectores de modelos distintos.
`modelRevision`	Fija la versión exacta del modelo. Por defecto usamos un commit de Hugging Face, no `main`.
`dimensions`	Confirma que el vector entra en la columna `vector(384)`.
`createdAt`	Permite saber cuándo se generó o regeneró la fila.

> **La primera vez puede tardar.** El archivo ONNX fp32 pesa unos **470 MB**. La descarga se guarda en `.cache/transformers/`, que está ignorada por git. Si cambiás de modelo o de revisión, se descarga otro conjunto de archivos. El texto de los documentos no se envía a Hugging Face; solo se descargan los archivos del modelo.

LA PERSISTENCIA

pgvector guarda vectores; Prisma guarda el resto

Prisma modela las entidades y relaciones habituales. El tipo `vector` de pgvector es una extensión de PostgreSQL que Prisma no trata como un escalar normal. Por eso el esquema declara la columna como `Unsupported("vector(384)")` y las operaciones vectoriales usan SQL parametrizado.

LA FICHA

DocumentChunk

- `id`
- `documentPageId`
- `chunkIndex`
- `content`
- `startChar`, `endChar`
- `embeddings` → N vectores

1 — N

LA COORDENADA

ChunkEmbedding

- `id`
- `chunkId`
- `modelId`
- `modelRevision`
- `dimensions`
- `embedding vector(384)`
- `createdAt`
- `unique(chunkId, modelId, modelRevision)`
- `onDelete: Cascade`

Por qué SQL parametrizado

Prisma Client no puede leer ni escribir el campo `embedding`. Las inserciones y búsquedas usan `Prisma.sql` con `$executeRaw` y `$queryRaw`. El vector se envía como texto y se castea con `::vector`. Nunca se concatena texto del usuario en la consulta y no se usa `$queryRawUnsafe`.

```
INSERT INTO "ChunkEmbedding" (
  "id", "chunkId", "modelId",
  "modelRevision", "dimensions", "embedding"
)
VALUES ($1, $2, $3, $4, $5, $6::vector)
ON CONFLICT ("chunkId", "modelId", "modelRevision")
DO UPDATE SET
  "dimensions" = EXCLUDED."dimensions",
  "embedding" = EXCLUDED."embedding",
  "createdAt" = CURRENT_TIMESTAMP;
```

La migración habilita la extensión

La migración empieza con `CREATE EXTENSION IF NOT EXISTS vector` y después crea la tabla. La columna es `vector(384)`, no `vector` sin tamaño: PostgreSQL rechaza un vector de otra dimensión, y esa es una protección útil.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE "ChunkEmbedding" (
  "id" TEXT NOT NULL,
  "chunkId" TEXT NOT NULL,
  "modelId" TEXT NOT NULL,
  "modelRevision" TEXT NOT NULL,
  "dimensions" INTEGER NOT NULL,
  "embedding" vector(384) NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "ChunkEmbedding_pkey" PRIMARY KEY ("id")
);

CREATE UNIQUE INDEX "ChunkEmbedding_chunkId_modelId_modelRevision_key"
  ON "ChunkEmbedding"("chunkId", "modelId", "modelRevision");

ALTER TABLE "ChunkEmbedding"
  ADD CONSTRAINT "ChunkEmbedding_chunkId_fkey"
  FOREIGN KEY ("chunkId") REFERENCES "DocumentChunk"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
```

La red de seguridad: el unique impide dos vectores del mismo chunk, modelo y revisión. La cascada impide vectores huérfanos cuando se borra un chunk, una página o un documento. La dimensión fija impide guardar un vector de otro tamaño.

Por qué no JSON: podrías guardar 384 números en un JSON, pero PostgreSQL no sabría compararlos con `<=>`, no tendría `vector_dims` y no podría usar índices vectoriales. pgvector convierte la lista en un tipo con operaciones reales. Tampoco agregamos HNSW/IVFFlat todavía: el corpus es chico y la búsqueda exacta es más fácil de inspeccionar.

LA GENERACIÓN

Generar embeddings sin duplicar

La acción se puede ejecutar muchas veces. Repetirla no debe crear una pila de vectores para el mismo fragmento. Esa propiedad se llama **idempotencia**, y acá tiene dos capas.

NAVEGADOR

1. Generar o regenerar

El panel del documento envía `{ "regenerate": false }` para completar pendientes o `{ "regenerate": true }` para rehacer todos.

↓

SERVIDOR · LECTURA

2. Buscar los chunks

Se leen todos los fragmentos del documento. Si no hay ninguno, responde **409 DOCUMENT_WITHOUT_CHUNKS**.

↓

SERVIDOR · COMPATIBILIDAD

3. Omitir los existentes

Sin regeneración, se buscan las filas que ya tienen el mismo `modelId`, `modelRevision` y `dimensions`. Esos chunks no se vuelven a procesar.

↓

SERVIDOR · LOTES

4. Inferencia por lote

Los pendientes se procesan en lotes de `EMBEDDING_BATCH_SIZE` (4 por defecto). Cada lote recibe sus prefijos y devuelve sus vectores.

↓

POSTGRESQL · UPSERT

5. Guardar con ON CONFLICT

Una sola sentencia inserta el lote. Si ya existe la clave, actualiza el vector en lugar de duplicarlo.

↓

NAVEGADOR · RESULTADO

6. Mostrar conteos

La UI muestra generados, omitidos, fallidos y el conteo final. Si un lote falló, el estado es `partial`; si fallaron todos, `failed`.

Idempotencia en dos capas

CAPA	QUÉ EVITA	DÓNDE ESTÁ
Omisión previa	Volver a generar filas que ya existen para el mismo modelo y revisión.	`findExistingChunkIds`
ON CONFLICT	Duplicados por una carrera o por una repetición inesperada. Actualiza la fila.	`upsertEmbeddingBatch`
Regeneración explícita	Que se reescriban vectores existentes sin que la persona lo pida.	`{ "regenerate": true }`

Fallos parciales

Un documento puede tener muchos lotes. Si uno falla, no queremos mentir diciendo que todo salió bien, pero tampoco queremos tirar el trabajo de los lotes que sí funcionaron. El resultado tiene tres estados:

ESTADO	CUÁNDO APARECE	QUÉ DEBERÍA HACER LA PERSONA
`complete`	Todos los chunks pendientes se generaron o se omitieron sin errores.	Nada. El documento está listo para buscar.
`partial`	Al menos un lote falló y al menos otro se guardó.	Revisar los lotes fallidos y volver a intentar. Los vectores anteriores de los lotes que fallaron siguen intactos.
`failed`	Ningún lote se pudo guardar.	Revisar la terminal: puede ser el modelo, la conexión o la base.

Los errores incluyen el número de lote y la etapa: `embedding` si falló la inferencia, `persistence` si falló PostgreSQL. La respuesta del API nunca incluye el texto de los documentos.

> **Regenerar chunks borra sus embeddings.** La cascada de `ChunkEmbedding → DocumentChunk` elimina los vectores cuando los fragmentos viejos se reemplazan. Después de regenerar fragmentos hay que volver a generar embeddings. No es un bug: es la consecuencia de que el vector pertenezca a un chunk concreto.

LA BÚSQUEDA

Buscar por significado, no por coincidencia

La pantalla `/search` recibe una pregunta, genera su embedding con el prefijo `query:` y busca los fragmentos más cercanos. Todavía no hay una respuesta redactada: solo los fragmentos que el sistema considera relevantes.



SERVIDOR · QUERY

4. Generar el vector de la pregunta

`embedQuery` agrega `query:`, ejecuta el modelo y valida 384 dimensiones finitas.

↓

POSTGRESQL · PGVECTOR

5. Buscar y ordenar

La consulta filtra por modelo, revisión y dimensión, ordena por `<=>` ascendente y limita a `topK`. No se aplica un umbral.

↓

NAVEGADOR · RESULTADOS

6. Mostrar documento y página

Cada resultado muestra contenido, documento, página, distancia y similitud. No se devuelve el vector completo.

La consulta SQL

```
SELECT
  c.id            AS "chunkId",
  c.content       AS "content",
  p."pageNumber"  AS "pageNumber",
  d."originalFilename" AS "documentName",
  e.embedding <=> $1::vector AS "distance"
FROM "ChunkEmbedding" e
JOIN "DocumentChunk" c ON c.id = e."chunkId"
JOIN "DocumentPage" p ON p.id = c."documentPageId"
JOIN "Document" d ON d.id = p."documentId"
WHERE e."modelId" = $2
  AND e."modelRevision" = $3
  AND e.dimensions = $4
ORDER BY e.embedding <=> $1::vector ASC
LIMIT $5;
```

El vector de la pregunta se envía como parámetro y se castea con `::vector`. El modelo, la revisión, la dimensión y el `topK` también son parámetros. La consulta no concatena texto del usuario.

Qué se filtra y qué se devuelve

FILTRO O CAMPO	POR QUÉ
`modelId`	Evita comparar vectores de modelos distintos.
`modelRevision`	Evita mezclar versiones del mismo modelo.
`dimensions`	Confirma que el vector entra en la columna y es comparable.
`topK`	Limita la cantidad de resultados. Por defecto 5; máximo 20.
Distancia	Se muestra para inspeccionar la recuperación. No se usa como umbral.
Similitud	Se calcula como `1 − distancia` para que sea más legible.
Vector	**No** se devuelve en la API ni se muestra en la interfaz.

> **La regla de oro:** si no hay embeddings del modelo actual, la búsqueda devuelve cero resultados. No mezcla vectores viejos, no inventa un modelo por defecto y no descarga el modelo solo para buscar en un índice vacío.

LA INTERFAZ

Qué ves en la pantalla

El detalle del documento conserva el texto y los fragmentos del Sprint 2 y agrega un panel de embeddings. La pantalla de búsqueda es un mostrador de diagnóstico: no redacta, solo muestra qué recuperó.

```
Documento: documento-de-prueba.pdf          [ Regenerar todos ]

Embeddings
Modelo Xenova/multilingual-e5-small · revisión 761b726dd34f · 384 dimensiones
2 de 2 fragmentos con embeddings
El modelo corre localmente. La primera ejecución puede descargar cientos de MB.

Búsqueda
Pregunta: ¿Cuál es el período de garantía del producto?     topK: [5]
[ Buscar fragmentos ]

#1  documento-de-prueba.pdf   Página 1   distancia 0.1366   similitud 86.3%
    El período de garantía del producto es de dos años.
#2  documento-de-prueba.pdf   Página 2   distancia 0.2238   similitud 77.6%
    El asistente divide el texto en fragmentos y conserva los metadatos.
```

ELEMENTO	QUÉ MUESTRA
Panel de embeddings	Modelo, revisión, dimensiones, conteo y botón para generar o regenerar.
Generar embeddings	Completa solo los fragmentos que no tienen vector para el modelo actual.
Regenerar todos	Vuelve a generar todos los vectores del documento. Es una acción explícita.
Estado de carga	Avisa que la primera ejecución puede descargar el modelo y tardar.
Estado parcial	Muestra cuántos se generaron, cuántos fallaron y en qué lote.
Pantalla de búsqueda	Pregunta, `topK`, resultados con documento, página, distancia y similitud.
Sin índice	«Todavía no hay embeddings para este modelo. Generá embeddings desde un documento.»
Sin resultados	Mensaje para probar otra pregunta. No se inventa una respuesta.

Cómo probarlo con el PDF de prueba

Usá `samples/documento-de-prueba.pdf`. Tiene dos páginas con las frases que aparecen en los resultados de ejemplo. Si el documento ya está en la biblioteca, no lo subas de nuevo: la aplicación no deduplica y crearías otro documento.

PREGUNTA	RESULTADO ESPERADO
«¿Cuál es el período de garantía del producto?»	Primero la **página 1**, con la frase de la garantía.
«¿Qué hace el asistente con el texto del documento?»	Primero la **página 2**, con la frase de los fragmentos y metadatos.
Pregunta vacía o de más de 1000 caracteres	`400 INVALID_SEARCH_QUERY`.
`topK` en 0, 21, `null` o texto	`400 INVALID_TOP_K`.
Otro `EMBEDDING_MODEL_ID` o revisión	`indexedChunkCount: 0` y `results: []` hasta regenerar.

> **La evidencia de este sprint:** la pregunta por la garantía devolvió primero la página 1 con distancia 0,1366; la pregunta por el texto del documento devolvió primero la página 2 con distancia 0,1368. Los resultados quedaron ordenados de menor a mayor distancia y sin vectores en la respuesta.

LO QUE TODAVÍA NO HACE

Los límites que conviene revisar

Ninguno de estos límites rompe la búsqueda vectorial. Son decisiones de alcance y puntos para el próximo sprint.

LÍMITE	QUÉ SIGNIFICA	CUÁNDO REVISARLO
Sin autenticación	La búsqueda es global sobre la biblioteca local. No hay usuarios ni permisos por documento.	Antes de exponer la app a Internet.
Sin OCR	Un PDF escaneado como imagen se rechaza porque no tiene texto seleccionable.	Cuando haya documentos escaneados reales.
Inferencia en CPU	La generación puede tardar varios segundos o minutos según la cantidad de chunks y el hardware.	Si el corpus crece o la latencia molesta.
Dependencia de red inicial	La primera ejecución descarga el modelo. Después usa la caché, aunque Transformers.js puede resolver metadatos remotos.	Si se despliega en un entorno sin salida a Internet.
Ventana de 512 tokens	El chunker mide caracteres, no tokens. Un fragmento largo puede truncarse en el modelo.	Al ajustar el tamaño de chunk con el tokenizador real.
Dimensión y precisión fijas	`vector(384)` y fp32. Cambiar de modelo o de precisión exige migración y regeneración.	Si se evalúa otro modelo de embeddings.
Sin umbral de distancia	Siempre se devuelven hasta `topK` resultados, aunque sean poco relevantes.	Cuando haya preguntas de evaluación y una métrica de calidad.
Sin índice ANN	La búsqueda exacta recorre todas las filas compatibles. Es O(n).	Cuando el corpus haga medible la latencia.
Sin reranking ni híbrida	La recuperación es solo vectorial; no combina BM25 ni reordena con otro modelo.	Después de medir la recuperación actual.
Sin respuesta generativa	La búsqueda no llama a un LLM. No hay chat ni citas redactadas.	Sprint 4.
Vectores viejos no se borran	Si cambiás el modelo, las filas anteriores quedan en la tabla para comparar.	Cuando se defina una política de limpieza.
Prisma Studio	No puede deserializar la columna `vector`; para inspeccionar la tabla se usa `psql`.	Si se necesita una consola visual.

ALCANCE

No hay LLM todavía

El Sprint 3 termina en la recuperación. La respuesta generativa es el siguiente paso, no una parte escondida de este.

RENDIMIENTO

Exacto antes que aproximado

HNSW e IVFFlat quedan para cuando el corpus justifique medir la diferencia. Primero se aprende con el orden exacto.

EL MAPA DE ARCHIVOS

Dónde vive cada parte de la búsqueda

Si mañana tenés que explicar el Sprint 3, este es el orden en el que conviene abrir los archivos.

```
src/
├── app/
│   ├── api/
│   │   ├── documents/[id]/embeddings/route.ts  ← generar vectores
│   │   └── search/route.ts                     ← buscar fragmentos
│   ├── documents/[id]/page.tsx                 ← detalle + panel
│   └── search/page.tsx                         ← pantalla de búsqueda
├── components/
│   ├── documents/
│   │   ├── DocumentEmbeddingsPanel.tsx         ← generar/regenerar
│   │   ├── DocumentChunkViewer.tsx             ← fragmentos + evento
│   │   └── DocumentDetail.tsx                  ← composición
│   └── search/
│       └── SearchPanel.tsx                     ← formulario y resultados
├── lib/
│   ├── embeddings/
│   │   ├── config.ts                           ← modelo, lotes y límites
│   │   ├── vector.ts                           ← 384, NaN, pgvector
│   │   ├── provider-core.ts                    ← prefijos y pooling
│   │   ├── provider.ts                         ← carga lazy
│   │   ├── store.ts                            ← SQL parametrizado
│   │   ├── generate.ts                         ← idempotencia y lotes
│   │   ├── search.ts                           ← búsqueda coseno
│   │   ├── summary.ts                          ← conteo por documento
│   │   └── input.ts                            ← validación de entrada
│   └── client/read-error.ts                    ← errores seguros
└── tests/
    ├── chunking.test.mjs
    └── unit/
        ├── embeddings.test.ts
        ├── embeddings-boundaries.test.ts
        ├── embeddings-generate.test.ts
        └── embeddings-sql-params.test.ts

prisma/schema.prisma
prisma/migrations/20261112153534_add_chunk_embeddings/
```

Checklist del Sprint 3

- [ ] Podés generar embeddings para un documento con chunks.
- [ ] Cada vector tiene 384 dimensiones finitas y se guarda con su modelo y revisión.
- [ ] Repetir la generación no duplica filas.
- [ ] La regeneración explícita reemplaza los vectores existentes.
- [ ] Un fallo parcial se reporta como `partial`, no como éxito total.
- [ ] La búsqueda filtra por modelo, revisión y dimensión.
- [ ] Los resultados se ordenan por distancia ascendente y muestran documento y página.
- [ ] La pregunta vacía, larga o con `topK` inválido se rechaza.
- [ ] No se devuelven vectores en la API ni en la interfaz.
- [ ] No hay respuesta generativa, reranking, búsqueda híbrida ni índice ANN.

COMANDOS Y LECTURA

Cómo probarlo y en qué orden leer

Cómo probarlo

```
npm run dev
npm test
npm run typecheck
npm run lint

# Generar embeddings:
curl -X POST http://localhost:3000/api/documents/<id>/embeddings \
  -H "Content-Type: application/json" -d '{}'

# Regenerar todos:
curl -X POST http://localhost:3000/api/documents/<id>/embeddings \
  -H "Content-Type: application/json" -d '{"regenerate":true}'

# Buscar:
curl -X POST http://localhost:3000/api/search \
  -H "Content-Type: application/json" \
  -d '{"query":"¿Cuál es el período de garantía?","topK":5}'
```

Para revisar el código

1. `src/lib/embeddings/vector.ts` — validación, formato pgvector y similitud.
2. `src/lib/embeddings/provider-core.ts` — prefijos, pooling y normalización.
3. `src/lib/embeddings/provider.ts` — carga lazy y singleton.
4. `src/lib/embeddings/store.ts` — SQL parametrizado, upsert y búsqueda.
5. `src/lib/embeddings/generate.ts` — lotes, idempotencia y fallos parciales.
6. `src/lib/embeddings/search.ts` — validación y búsqueda exacta.
7. `prisma/schema.prisma` — relaciones, unique y cascada.
8. `src/components/search/SearchPanel.tsx` — estados de la UI.

PARA COMPROBAR QUE ENTENDISTE

Seis preguntas de repaso

Estas son las preguntas que conviene poder responder sin mirar el código. Si salen, entendiste los embeddings, la distancia y el filtro de compatibilidad.

1. ¿Qué representa un embedding y por qué no es una palabra clave?

Representa el significado de un texto como una coordenada numérica. Una palabra clave compara coincidencias exactas; un embedding compara cercanía semántica, aunque se usen sinónimos o distinta redacción. No es una traducción legible: es una posición en un espacio vectorial.

2. ¿Qué diferencia hay entre similitud coseno y distancia coseno?

La similitud es el coseno del ángulo: 1 es «misma dirección», 0 es perpendicular. La distancia es `1 − similitud`: 0 es idéntico y valores mayores son más lejanos. pgvector `<=>` devuelve distancia; por eso ordenamos ascendente.

3. ¿Por qué usamos `passage:` para documentos y `query:` para preguntas con E5?

Porque el modelo fue entrenado con esos prefijos. Le indican qué tipo de texto está viendo y ayudan a que la comparación funcione como fue entrenada. No son opcionales: omitirlos degrada la recuperación.

4. ¿Por qué no conviene comparar embeddings generados con modelos distintos?

Porque cada modelo construye su propio espacio vectorial. Dos vectores de modelos distintos pueden parecer cercanos sin significar lo mismo. Por eso guardamos `modelId`, `modelRevision` y `dimensions`, y filtramos por ellos.

5. ¿Qué aporta pgvector sobre guardar una lista de números como JSON?

Un tipo nativo `vector`, operadores de distancia como `<=>`, funciones como `vector_dims` y la posibilidad de agregar índices ANN. En JSON podrías guardar los números, pero PostgreSQL no sabría compararlos semánticamente.

6. ¿Por qué empezamos con búsqueda exacta antes de agregar HNSW?

Porque es correcta, simple y fácil de inspeccionar. HNSW intercambia algo de exactitud por velocidad. Primero hay que medir si el corpus y la latencia realmente lo necesitan; agregarlo antes sería optimizar sin datos.

> **La respuesta corta:** el Sprint 3 convierte cada fragmento en una coordenada, guarda esa coordenada con su modelo y su revisión, y busca las coordenadas más cercanas a una pregunta. Todavía no redacta la respuesta; primero aprende a encontrar los fragmentos correctos.

GLOSARIO

Los términos que conviene tener a mano

Embedding

Una lista de números que representa el significado de un texto. En este proyecto tiene 384 dimensiones y se usa para comparar fragmentos con preguntas.

Vector

La lista de números en sí. Se guarda en PostgreSQL con el tipo `vector(384)` de pgvector.

Similitud coseno

El coseno del ángulo entre dos vectores. Vale 1 para la misma dirección, 0 para vectores perpendiculares y −1 para direcciones opuestas.

Distancia coseno

El complemento de la similitud: `1 − similitud`. pgvector la calcula con `<=>`. Menor distancia significa mayor similitud.

pgvector

Una extensión de PostgreSQL que agrega el tipo `vector`, operadores de distancia y funciones para inspeccionar vectores.

E5

La familia de modelos de embeddings usada en el sprint. La variante `multilingual-e5-small` produce 384 dimensiones y espera los prefijos `passage:` y `query:`.

Mean pooling

Promediar los vectores de todos los tokens de un texto para obtener un único vector. Es la estrategia que usa el pipeline configurado.

Normalización L2

Escalar el vector para que su longitud sea 1. Con vectores normalizados, el producto punto coincide con la similitud coseno.

topK

La cantidad máxima de resultados que devuelve la búsqueda. Por defecto 5; máximo 20 en este sprint.

Distancia exacta

Comparar la pregunta contra todos los vectores compatibles y ordenar. No usa un índice aproximado. Es O(n).

HNSW / IVFFlat

Índices aproximados de pgvector. Intercambian algo de exactitud por velocidad. No se usan en este sprint.

Idempotencia

La propiedad de una operación que, ejecutada varias veces, produce el mismo estado final. Generar embeddings es idempotente por la omisión previa y el `ON CONFLICT`.

Fallo parcial

Un resultado donde algunos lotes se guardaron y otros no. Se reporta como `partial`; nunca como éxito total.

RAG

Retrieval-Augmented Generation: recuperar fragmentos relevantes y usarlos como contexto para generar una respuesta. El Sprint 3 cubre la recuperación; el Sprint 4 agrega la generación.

> **La frase para llevarte:** el Sprint 1 construyó el archivo. El Sprint 2 construyó el índice de fichas. El Sprint 3 le dio a cada ficha una coordenada y construyó el mapa para encontrarla. Todavía no hay redactor, pero ya hay bibliotecario: una pregunta puede recuperar los fragmentos correctos y decir de qué documento y de qué página salieron.